

```
package jp.co.active.kenshu.diary;

import java.io.IOException;
import java.time.DateTimeException;
import java.time.LocalDate;
import java.util.Map;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

/**
 * 日記確認画面を表示するサーブレット
 */
@WebServlet("/calendar/diary/confirm")
public class ConfirmServlet extends HttpServlet {

    /**
     * 入力画面から送られた内容をチェックし、確認画面へ遷移する
     */
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");

        String year = request.getParameter("year");
        String month = request.getParameter("month");
        String day = request.getParameter("day");

        String title = request.getParameter("title");
        String main = request.getParameter("main");
        String feel = request.getParameter("feel");
        String[] weather = request.getParameterValues("weather");

        // タイトル・本文・気分・天気チェック
        Map<String, String> errors = DiaryValidationHelper.validateDiary(title, main, feel, weather);
```

```
LocalDate targetDate = null;

try {
    // 年月日チェック
    targetDate = DiaryValidationHelper.createDate(year, month, day);

    // 年月日が未来日かチェック
    if (targetDate.isAfter(LocalDate.now())) {
        errors.put("date", "未来の日付の日記は入力できません。");
    }

} catch (NumberFormatException e) {
    // 年月日が数字以外の場合
    errors.put("date", "年月日は数字で指定してください。");

} catch (DateTimeException e) {
    // 2024年2月30日など、存在しない年月日の場合
    errors.put("date", "存在しない年月日です。");

} catch (IllegalArgumentException e) {
    // 年月日がnullまたは空文字の場合
    errors.put("date", e.getMessage());
}

// JSPで再表示するために値をセットする
setRequestValues(request, year, month, day, title, main, feel, weather, targetDate);

// エラーがある場合は入力画面へ戻す
if (!errors.isEmpty()) {
    request.setAttribute("errors", errors);
    request.getRequestDispatcher("/WEB-INF/jsp/diary/diary.jsp").forward(request,
response);
    return;
}

// エラーがない場合は確認画面へ進む
request.getRequestDispatcher("/WEB-INF/jsp/diary/confirm.jsp").forward(request,
response);
}

/**
```

```
* JSPで使う値をrequestにセットする
*
* @param request リクエスト
* @param year 年
* @param month 月
* @param day 日
* @param title タイトル
* @param main 本文
* @param feel 今日の気分
* @param weather 今日の天気
* @param targetDate チェック済みの日付
*/
```

```
private void setRequestValues(
    HttpServletRequest request,
    String year,
    String month,
    String day,
    String title,
    String main,
    String feel,
    String[] weather,
    LocalDate targetDate) {

    // 日付が正しい場合はLocalDateから値をセットする
    if (targetDate != null) {
        request.setAttribute("year", targetDate.getYear());
        request.setAttribute("month", targetDate.getMonthValue());
        request.setAttribute("day", targetDate.getDayOfMonth());

        // 日付エラーの場合は、入力画面に戻すために元の値をセットする
    } else {
        request.setAttribute("year", year);
        request.setAttribute("month", month);
        request.setAttribute("day", day);
    }

    request.setAttribute("title", title);
    request.setAttribute("main", main);
    request.setAttribute("feel", feel);
    request.setAttribute("weather", weather);
}
```

```
request.setAttribute("shineChecked", DiaryValidationHelper.contains(weather, "晴"));
request.setAttribute("cloudChecked", DiaryValidationHelper.contains(weather, "曇"));
request.setAttribute("rainChecked", DiaryValidationHelper.contains(weather, "雨"));
request.setAttribute("snowChecked", DiaryValidationHelper.contains(weather, "雪"));
    }
}
```